

AI VIDEO ASSISTANT

A Modular System for Video Transcription, Meeting Summarization, Action-Item Extraction, Decision Mining, and Retrieval-Augmented Question Answering

Research / Technical Project Report
Implementation analyzed from the public repository: emManab/video_asst
Application layer: Python + Streamlit
Primary LLM: Mistral Small (model identifier used in source: mistral-small-latest)
Speech recognition: OpenAI Whisper (English) / Sarvam Saaras v2.5 (Hinglish translation path)
Retrieval: ChromaDB + all-MiniLM-L6-v2 embeddings

Prepared as a code-grounded technical research paper. Model names, parameters, file structure, and processing stages are taken from the repository source code and README; no training of a new model is claimed.

Abstract

This paper presents a detailed technical analysis of an AI Video Assistant implemented as a modular Python application. The system converts long-form video or audio into a searchable knowledge source by combining audio preprocessing, automatic speech recognition (ASR), large-language-model (LLM) analysis, structured information extraction, and retrieval-augmented generation (RAG). The application accepts YouTube URLs as well as uploaded local media. For YouTube, it first attempts to retrieve an English transcript directly; if a transcript is unavailable, the system falls back to downloading audio with yt-dlp. Local media is converted to mono 16-kHz WAV audio and divided into ten-minute chunks. English speech is transcribed locally using OpenAI Whisper, while the Hinglish path uses Sarvam's speech-to-text translation API. The resulting transcript is passed to Mistral Small for title generation, map-reduce summarization, action-item extraction, decision extraction, and open-question detection. For interactive questioning, the transcript is split into smaller passages, embedded with the Hugging Face/Sentence Transformers model all-MiniLM-L6-v2, stored in a persistent ChromaDB collection, and retrieved through a LangChain LCEL pipeline before Mistral generates a grounded answer. The architecture deliberately separates speech recognition, semantic analysis, retrieval, and user interface concerns, making the project suitable as an educational reference implementation for multimodal meeting intelligence.

Keywords: video understanding, speech-to-text, Whisper, Mistral AI, Sarvam AI, RAG, ChromaDB, Sentence Transformers, LangChain, Streamlit, meeting intelligence

1. Introduction

Meetings, lectures, interviews, presentations, and other spoken-video resources contain valuable information, but their usefulness is limited when users must watch the entire recording to find one fact. A practical assistant therefore needs more than transcription. It must transform raw speech into a compact representation, identify decisions and tasks, and provide a question-answering interface that can locate relevant parts of the transcript. The analyzed project addresses this problem with a pipeline rather than a single monolithic model. Its README explicitly describes support for YouTube URLs and local media, English Whisper transcription, Hinglish-to-English Sarvam transcription, Mistral-based summarization and extraction, and ChromaDB-based RAG chat. This separation is important because each component solves a different subproblem and can be replaced independently. The repository's architecture lists app.py as the Streamlit entry point, main.py as the CLI entry point, and dedicated modules for transcription, summarization, extraction, retrieval, vector storage, and audio processing.

2. Problem Statement

Given a video or audio recording containing spoken information, build an application that can: (1) acquire or ingest the media; (2) convert speech into usable English text; (3) generate a concise professional summary; (4) identify actionable tasks, owners and deadlines; (5) identify decisions; (6) identify unresolved questions; and (7) answer natural-language questions using only the information present in the source transcript. The last requirement is particularly important because a generic LLM can answer using prior knowledge or hallucinated information. The project instead constructs a local vector index of the transcript and explicitly instructs the answer-generation model to use retrieved transcript context.

3. Objectives

Objective	Implementation
Flexible input	YouTube URL or local audio/video upload.
Robust audio handling	yt-dlp for YouTube fallback, pydub for conversion/chunking, FFmpeg for media codecs.

English ASR	OpenAI Whisper loaded locally; model size controlled by WHISPER_MODEL, default small.
Hinglish ASR + translation	Sarvam speech-to-text-translate endpoint using saaras:v2.5 in the current repository.
Long-context summarization	RecursiveCharacterTextSplitter + Mistral map-reduce summarization.
Meeting intelligence	Mistral chains for actions, decisions and unresolved questions.
Semantic retrieval	all-MiniLM-L6-v2 embeddings + persistent ChromaDB.
Grounded Q&A;	Similarity retrieval (k=4) followed by Mistral answer generation.
User experience	Streamlit dashboard with processing status, results cards, transcript viewer and chat.

4. System Architecture

The system follows a staged pipeline. The application layer is implemented in Streamlit, while the core processing is divided into independent Python modules. The repository README describes the high-level flow as Input → Audio Processing → Transcription → LLM Analysis + RAG Build → Streamlit Dashboard.

4.1 End-to-End Pipeline

Stage	Input	Main implementation	Output
1. Acquisition	URL / uploaded file	process_input()	Transcript object or audio chunks
2. Audio processing	Media	yt-dlp / pydub / FFmpeg	16-kHz mono WAV chunks
3. Transcription	Audio chunks	Whisper or Sarvam	Unified English transcript
4. Title	Transcript	Mistral Small	Short title
5. Summary	Transcript chunks	Mistral map-reduce	Professional bullet summary
6. Extraction	Transcript	Mistral chains	Actions / decisions / questions
7. Indexing	Transcript	MiniLM embeddings + ChromaDB	Persistent vector collection
8. Q&A;	Question + retrieved chunks	LangChain LCEL + Mistral	Grounded answer
9. Presentation	Results	Streamlit	Interactive dashboard

Video / Audio	→	Audio Processing yt-dlp / pydub / FFmpeg	→	ASR Whisper / Sarvam
Transcript	→	Mistral Analysis Title + Summary + Extraction	↘	Streamlit Results
			↘	RAG Chat MiniLM → Chroma → Retriever → Mistral

Figure 1. Conceptual architecture reconstructed from the repository's module structure and runtime pipeline.

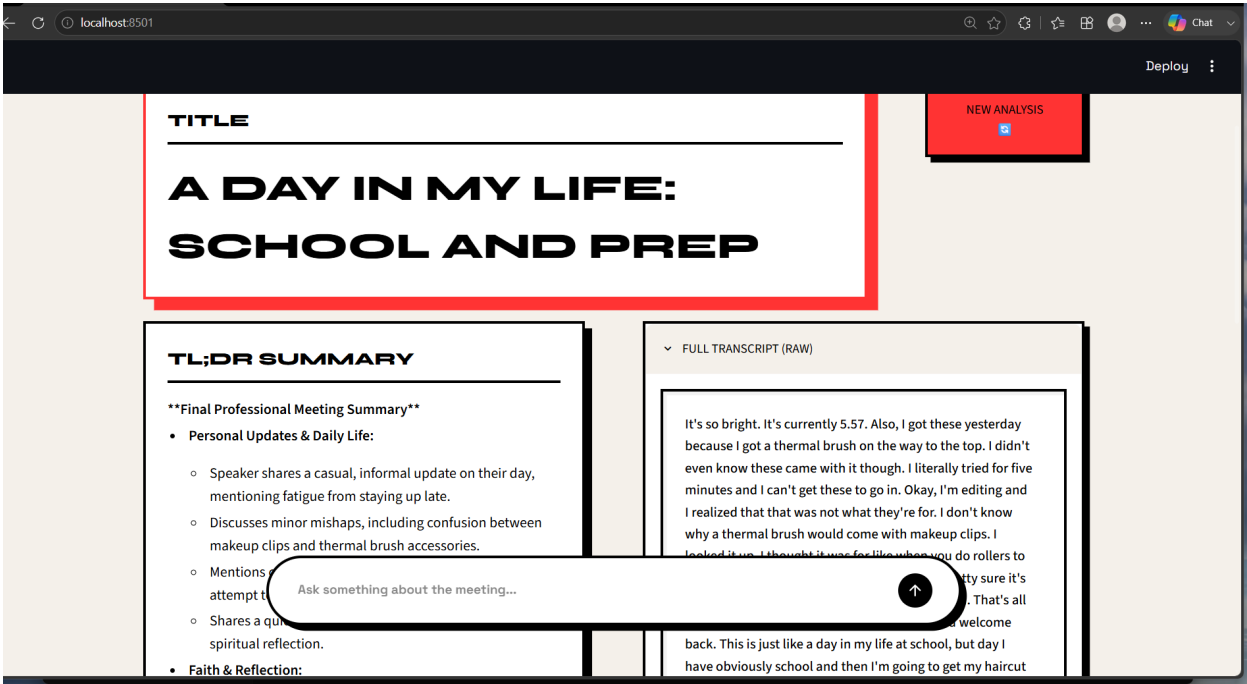


Figure 2. Local Streamlit interface produced by the analyzed application. The dashboard exposes the generated title, summary, raw transcript, extracted meeting intelligence, and RAG chat.

5. Input Acquisition and Audio Processing

The audio processor is the first major engineering layer. The function `process_input()` detects whether the source is a URL or a local path. For a YouTube URL, the implementation first attempts to retrieve captions using the YouTube Transcript API. This is an optimization: if captions are available, the system can bypass media downloading and Whisper transcription entirely. If transcript retrieval fails, the code falls back to `yt-dlp` and extracts audio to WAV using `FFmpeg`. For a local file, `pydub` converts the source to mono, 16-kHz WAV. The resulting audio is then split into ten-minute chunks.

```
# Simplified representation of the repository logic
if is_url:
    text = fetch_youtube_transcript(source)
    if text:
        return TextTranscript(text)
    wav_path = download_youtube_audio(source)
else:
    wav_path = convert_to_wav(source)

chunks = chunk_audio(wav_path, chunk_minutes=10)
return chunks
```

The conversion step is technically important because speech-recognition models work best when audio is normalized to a consistent representation. The repository explicitly sets one channel and a 16-kHz sample rate before exporting WAV.

6. Speech Recognition

6.1 English: OpenAI Whisper

For English, the code imports the `openai-whisper` package and lazily loads a model using `whisper.load_model(WHISPER_MODEL)`. The default is `small`, although the environment variable supports `tiny`, `base`, `small`, `medium` and `large`. The model performs local inference through `model.transcribe()`. Whisper is a Transformer sequence-to-sequence ASR system trained on a large and diverse multilingual dataset; OpenAI describes the training corpus as approximately 680,000 hours of multilingual and multitask supervised data. The project's use of local Whisper is therefore a deliberate choice for English: after the model is downloaded, transcription does not require a speech API call.

6.2 Hinglish: Sarvam Saaras v2.5

The repository provides a separate Hinglish route. It calls Sarvam's legacy `/speech-to-text-translate` endpoint with model identifier `saaras:v2.5`. The code further splits each ten-minute chunk into 25-second WAV pieces because the synchronous API path has a short per-request duration limit. Each response returns English text, and the pieces are concatenated. This is useful for code-mixed Hindi-English speech because the downstream LLM pipeline receives a single English transcript.

Important reproducibility note: Sarvam's current documentation marks Saaras v2.5 and the dedicated `speech-to-text-translate` endpoint as `legacy/deprecated-soon` and recommends Saaras v3 with the standard `speech-to-text` endpoint and `mode='translate'`. This does not change what the current repository implements; it is a future maintenance item.

6.3 Transcript Fast Path

The transcriber contains a useful optimization: if the audio processor returns a `TextTranscript` object, `transcribe_all()` returns the transcript directly and does not load Whisper. Consequently, a YouTube video with usable English captions can avoid both audio download and local ASR. This makes the architecture adaptive rather than forcing every input through the most expensive path.

7. LLM-Based Semantic Analysis

The semantic reasoning layer uses LangChain's Mistral integration, ChatMistralAI. The source code consistently specifies the model identifier mistral-small-latest. Different tasks use slightly different temperatures: summarization and RAG use 0.3, while extraction uses 0.2. Lower temperature is appropriate for extraction because the desired behavior is structured and conservative rather than creative.

7.1 Title Generation

The title generator sends only the first 2,000 characters of the transcript to Mistral and instructs it to produce a short professional title of at most eight words. Restricting the title context is computationally efficient because a title generally does not require the entire recording.

```
title_chain =
    RunnablePassthrough()
    | RunnableLambda(lambda x: {"text": x})
    | ChatPromptTemplate(...)
    | Llm
    | StrOutputParser()

return title_chain.invoke(transcript[:2000])
```

7.2 Map-Reduce Summarization

The summarizer implements a classic map-reduce strategy. The transcript is first split with RecursiveCharacterTextSplitter using a 3,000-character chunk size and 200-character overlap. Each chunk is independently summarized by Mistral. The partial summaries are concatenated and passed to a second Mistral prompt that asks for a single professional bullet-point meeting summary. The benefit is that long transcripts can be processed in manageable pieces rather than requiring the entire raw transcript in one prompt.

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=3000,
    chunk_overlap=200
)

chunk_summaries = [
    map_chain.invoke({"text": chunk})
    for chunk in chunks
]

combined = "\n\n".join(chunk_summaries)
final_summary = combined_chain.invoke(combined)
```

7.3 Action Items, Decisions, and Open Questions

The extraction module defines one generic chain builder and three task-specific prompts. Action-item extraction requests task description, owner and deadline; missing deadlines are explicitly represented as 'Not specified'. A second chain extracts key decisions, and a third extracts unresolved questions or topics requiring follow-up. This design reduces duplication because the LLM setup and prompt wiring are centralized in build_chain().

8. Retrieval-Augmented Generation (RAG)

The RAG subsystem addresses a different problem from summarization. Summarization compresses the meeting into a human-readable overview; RAG preserves access to details that may not appear in the summary. The implementation builds a persistent ChromaDB collection from the transcript, then retrieves the four most similar transcript chunks for each user question.

8.1 Chunking and Embedding

The vector-store module uses RecursiveCharacterTextSplitter with a 500-character chunk size and 50-character overlap. Each chunk becomes a LangChain Document with a chunk_index metadata field. The

embedding model is explicitly set to sentence-transformers/all-MiniLM-L6-v2 and is forced to run on CPU. The model maps sentences and short paragraphs into a 384-dimensional dense vector space and is designed for semantic similarity and search. This makes it an economical local embedding model for a small meeting knowledge base.

8.2 ChromaDB Persistence and Similarity Search

The vector store persists to a local directory named `vector_db` and uses a collection called `meeting_transcript`. The retriever is configured for similarity search with `k=4`. At query time, the user's question is embedded using the same embedding function and compared with the stored transcript vectors. Chroma returns the most semantically similar passages, which are concatenated into a context string.

8.3 Grounded Answer Generation

The retrieved passages and the user's question are then passed through an LCEL chain into Mistral. The system prompt explicitly says to answer only from the meeting transcript context and to return a fixed fallback sentence when the answer is absent. This is the project's primary anti-hallucination mechanism. It is not a mathematical guarantee of factuality, but it constrains the model to the retrieved evidence supplied by the application.

```
rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)
```

9. Application and User Interface

The Streamlit application is implemented in `app.py`. It loads environment variables from `.env` for local development and can also read Streamlit secrets for hosted execution. The interface accepts either a YouTube URL or an uploaded audio/video file and exposes English/Hinglish selection. After the user presses the analysis button, the application stores the uploaded file in a temporary path, sets processing state, and reruns the Streamlit script.

The runtime pipeline in `app.py` is explicit and easy to audit: `process_input` → `transcribe_all` → `generate_title` → `summarize` → `extract_action_items` → `extract_key_decisions` → `extract_questions` → `build_rag_chain`. The result is stored in Streamlit session state. The dashboard then renders the title, summary, raw transcript, three extraction panels, and an interactive chat input connected to the RAG chain.

10. Repository and Code Responsibilities

File / module	Responsibility	Key techniques
<code>app.py</code>	Web application and orchestration	Streamlit, session state, custom CSS, status pipeline
<code>main.py</code>	Command-line execution	Same processing pipeline + terminal chat loop
<code>test.py</code>	Quick smoke test	Hard-coded example URL, transcript and analysis checks
<code>utils/audio_processor.py</code>	Media acquisition and preprocessing	YouTube Transcript API, yt-dlp, pydub, FFmpeg
<code>core/transcriber.py</code>	Speech recognition routing	Whisper local inference, Sarvam REST calls
<code>core/summarizer.py</code>	Title + summary	Mistral Small, recursive splitting, map-reduce
<code>core/extractor.py</code>	Meeting intelligence	Prompt-based action/decision/question extraction
<code>core/vector_store.py</code>	Vector database	MiniLM embeddings, Chroma persistence, similarity retrieval
<code>core/rag_engine.py</code>	Question answering	Retriever + context formatting + Mistral LCEL chain
<code>requirements.txt</code>	Python dependency declaration	Whisper, Torch, LangChain, ChromaDB, Streamlit, etc.

11. Dependency Stack

The repository's `requirements.txt` contains `yt-dlp`, `pydub`, `ffmpeg-python`, `openai-whisper`, `torch`, `torchaudio`, `deep-translator`, LangChain core/community/Mistral integrations, Mistral's SDK, ChromaDB, Sentence Transformers, LangChain Hugging Face support, Hugging Face Hub, `tiktoken`, Streamlit, `watchdog`, `python-dotenv`, `NumPy`, `tqdm`, `requests`, `langchain-chroma`, and `youtube-transcript-api`. The stack therefore combines conventional media tooling with modern LLM/RAG infrastructure.

12. Why This Architecture Was Chosen

Modularity: Speech recognition, LLM reasoning, vector retrieval and UI are isolated in separate modules. This makes debugging and model replacement easier.

Cost control: English Whisper and MiniLM embeddings run locally. Only semantic LLM operations and the optional Hinglish transcription path require remote APIs.

Long-video handling: Audio and transcript chunking avoid forcing a complete recording through one operation.

Two complementary representations: The summary is optimized for human scanning, while the vector index preserves fine-grained searchable evidence.

Grounding: The RAG prompt explicitly limits answers to retrieved transcript context.

Usability: Streamlit makes the pipeline accessible without requiring a separate frontend/backend stack.

13. Algorithms and Data Flow

13.1 Transcription Algorithm

Algorithm TRANSCRIBE_ALL(chunks, language)

1. If chunks is a TextTranscript:
 return chunks.text
2. Select engine:
 Sarvam if language == "hinglish"
 otherwise Whisper
3. For each audio chunk:
 transcribe chunk
 append text to full transcript
4. Return normalized concatenated transcript

13.2 Map-Reduce Summarization Algorithm

Algorithm SUMMARIZE(transcript)

1. Split transcript into 3000-character passages with 200-character overlap.
2. For each passage:
 send to Mistral with concise-summary prompt.
3. Concatenate all partial summaries.
4. Send combined summaries to Mistral with professional-summary prompt.
5. Return final bullet-point summary.

13.3 RAG Question-Answering Algorithm

Algorithm ASK_QUESTION(question)

1. Embed the question with all-MiniLM-L6-v2.
2. Perform similarity search in ChromaDB.
3. Retrieve top k=4 transcript chunks.
4. Concatenate retrieved text into context.
5. Give context + question to Mistral.
6. Return concise answer grounded in context.

14. Complexity and Resource Considerations

Let T be the number of characters in the transcript, A the duration of audio, C the number of transcript chunks used for summarization, and R the number of retrieved chunks per question. Audio preprocessing is approximately linear in audio duration because the media must be decoded and traversed. Whisper inference is also fundamentally proportional to the amount of audio processed, with practical runtime strongly dependent on model size and CPU/GPU hardware. Map-reduce summarization requires approximately C LLM calls for the map phase plus one reduction call. Extraction adds three additional LLM calls. RAG indexing requires embedding each transcript passage once, while each chat query performs one query embedding plus a similarity search and one generation call. The current implementation therefore favors architectural simplicity and predictable stages over aggressive batching or asynchronous execution.

15. Model Selection Rationale

Model / system	Role	Why it fits this project	Execution
OpenAI Whisper — default 'small'	English ASR	Strong general-purpose speech recognition; local inference avoids per-minute ASR API dependency.	Local
Sarvam Saaras v2.5	Hinglish speech → English	Designed for Indian-language speech translation and code-mixed scenarios.	Cloud API

Mistral Small — 'mistral-small-latest'	Title, summary, extraction, RAG answer	General text reasoning/generation layer with one model reused across multiple tasks.	Cloud API
all-MiniLM-L6-v2	Transcript embeddings	Compact 384-dimensional sentence embeddings suitable for semantic search.	Local CPU
ChromaDB	Vector storage/retrieval	Local persistence and similarity search for transcript passages.	Local

16. Experimental / Functional Evaluation

The supplied application screenshot demonstrates a successful local Streamlit run and shows the expected result structure: a generated title, a TL;DR summary, a full raw transcript, and the interactive question box. The repository also contains a smoke-test script that exercises input processing, transcription, title generation, summarization and extraction. However, the repository does not contain a formal benchmark dataset, ground-truth annotations, measured Word Error Rate, ROUGE/BERTScore, retrieval recall, latency benchmark, or human evaluation protocol. Therefore, this report does not invent numerical accuracy results. The appropriate claim is that the project demonstrates functional integration of the components rather than a scientifically benchmarked model improvement.

16.1 Suggested Evaluation Protocol

For a future academic evaluation, a dataset of 20–100 English and Hinglish recordings could be annotated for transcript correctness, summary completeness, action-item precision/recall, decision extraction accuracy, open-question recall, and RAG answer faithfulness. For ASR, Word Error Rate (WER) and Character Error Rate (CER) should be measured. For retrieval, Recall@k and MRR can quantify whether the correct transcript segment appears among the retrieved passages. For generated summaries, ROUGE can be reported as a reference-based metric, but human ratings for factual consistency and usefulness should be included because meeting summaries are highly sensitive to semantic omissions. For RAG, a human or programmatic ground-truth set of questions should be used to measure answer correctness and citation/context faithfulness.

17. Limitations

1. No true visual understanding: Despite the project being called an AI Video Assistant, the core analysis operates on speech/transcript content. It does not analyze video frames, objects, slides, faces, OCR, gestures, or visual events.
2. Potential transcript errors: All downstream LLM outputs depend on transcription quality when captions are unavailable.
3. Prompt-based extraction: Action items and decisions are inferred by the LLM and are not validated against structured ground truth.
4. RAG granularity: The vector store uses character-based chunks rather than speaker-aware or timestamp-aware segments, so answers do not inherently return source timestamps.
5. Single collection design: The code uses one collection name, 'meeting_transcript', which can create lifecycle/isolation concerns if many meetings are processed in the same persistent directory.
6. API dependency: Mistral is required for the semantic-analysis stage; Hinglish mode additionally depends on Sarvam.
7. Legacy Sarvam integration: The repository currently uses Saaras v2.5's legacy translate endpoint; current Sarvam documentation recommends migration to Saaras v3.
8. Reproducibility of 'latest' model aliases: 'mistral-small-latest' is an alias rather than a pinned model version, so outputs can change as the provider updates the alias.

18. Security and Privacy Considerations

API keys are expected in a .env file and the repository includes .gitignore/.env.example conventions. Keys should never be committed to Git. The English Whisper transcription path can run locally, which is advantageous for privacy-sensitive recordings, but the transcript is subsequently sent to the Mistral API for title generation, summarization, extraction and RAG answers. Hinglish audio is also sent to Sarvam. For confidential meetings, users should understand this data flow and verify provider policies before processing sensitive material. A production implementation should add authentication, per-user data isolation, automatic cleanup of uploaded media, secure secret storage, and explicit retention controls.

19. Reproducibility and Local Execution

The repository specifies Python 3.10+ and FFmpeg on PATH. The documented environment variables are MISTRAL_API_KEY, optional SARVAM_API_KEY, WHISPER_MODEL (default small), and SARVAM_STT_MODEL (default saaras:v2.5). The Streamlit entry point is streamlit run app.py. The CLI entry point is python main.py. The first English Whisper run downloads the selected model, after which the model is cached locally. ChromaDB persists its local vector data under the repository's vector_db directory.

20. Recommended Improvements

A stronger second version could introduce: (1) timestamp-aware transcript segments so answers can link back to the video; (2) speaker diarization and speaker labels; (3) per-meeting Chroma collections or unique collection IDs; (4) asynchronous/batched LLM calls to reduce latency; (5) pinned model versions for reproducibility; (6) migration from Sarvam v2.5 to Saaras v3; (7) evaluation datasets and automated regression tests; (8) a visual-analysis branch using frame sampling and a vision-language model; (9) structured JSON/Pydantic outputs for extraction instead of free-form text; and (10) source citations in RAG responses showing which transcript chunks support each answer.

21. Conclusion

The analyzed AI Video Assistant is best understood as an integrated AI application rather than a newly trained machine-learning model. Its contribution is the engineering of a complete pipeline in which specialized components are composed into a useful workflow. Whisper provides local English speech recognition; Sarvam provides a Hinglish-to-English path; Mistral Small performs high-level language understanding; all-MiniLM-L6-v2 converts transcript passages into semantic vectors; ChromaDB stores and retrieves those vectors; LangChain connects retrieval to generation; and Streamlit turns the entire pipeline into an accessible application. The architecture demonstrates a practical division of labor: deterministic software handles media and data movement, specialized models handle speech and semantic representation, and an LLM handles language-level synthesis. The strongest technical idea is the combination of summarization and RAG: the summary gives a compact overview, while the vector index retains access to details that would otherwise be lost. For an academic project, the next major step is not simply adding more models, but establishing a reproducible evaluation framework and improving source grounding, timestamp support, multi-meeting isolation, and model version pinning.

22. References and Source Notes

1. emManab/video_asst — project repository, README, application and core modules. GitHub. The repository documents the architecture, environment variables, execution instructions and feature set; the source files provide the implementation details analyzed in this paper.
2. Radford, A. et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. This is the research basis for Whisper's large-scale weakly supervised speech recognition approach.
3. OpenAI. Introducing Whisper. 2022. Describes Whisper as a multilingual, multitask speech-recognition system trained on approximately 680,000 hours of data.
4. Mistral AI Documentation — model and API documentation. The repository currently calls the Mistral API through the LangChain Mistral integration using the model identifier mistral-small-latest.
5. Sarvam AI Documentation — Saaras. Current documentation describes Saaras v3/v4 as the recommended/current speech-to-text family and marks Saaras v2.5's speech-to-text-translate endpoint as legacy. The repository itself currently uses Saaras v2.5.
6. Sentence Transformers / Hugging Face model card. The repository uses sentence-transformers/all-MiniLM-L6-v2, a compact sentence embedding model producing 384-dimensional vectors for semantic search and similarity.
7. Chroma Documentation — embeddings and retrieval. Chroma provides local vector storage and similarity search and supports local embedding functions.
8. LangChain Documentation. The project uses prompt templates, text splitters, output parsers and LCEL runnable composition to construct summarization, extraction and RAG pipelines.

Repository: https://github.com/emManab/video_asst

Primary source files analyzed: app.py, main.py, test.py, core/transcriber.py, core/summarizer.py, core/extractor.py, core/rag_engine.py, core/vector_store.py, utils/audio_processor.py, requirements.txt.

Methodological note: Claims about what the project actually implements are grounded in its source code and README. External documentation is used only to explain the underlying models and technologies and to flag current compatibility/deprecation considerations.